Aula de hoje: movimentação do player e fase de testes

13/10/2026 · Instrutor Pedro Leonardo

O que vamos fazer hoje

Até agora o player andava sempre na mesma velocidade e pulava uma vez só. Hoje a movimentação ganha mecânicas novas, e vamos construir uma fase de testes para experimentar cada uma delas e ajustar os números até o controle ficar gostoso.

- **Andar e correr**: o player começa andando e, depois de um tempo, passa a correr. A velocidade pode subir aos poucos.
- **Zona de pulo**: uma parede que define quantos pulos o player tem: 0 (não pula), 1 (normal) ou 2 (pulo duplo).
- **Deslizamento**: o player acelera e freia de forma suave, em vez de arrancar e parar seco.
- **Coyote time**: o player ainda consegue pular logo depois de sair de uma borda.
- **Painel de informações**: um texto na tela que mostra velocidade, estado, pulos, coyote time e se o player está no chão.
- **Fase de testes**: uma cena só para testar e ajustar a movimentação.
- **Dicas de game design**: como usar tudo isso para deixar o jogo mais gostoso de jogar.

Os estados do movimento vão funcionar assim:

Parado → Andando → Correndo → Deslizando → Parado

Lembrando a regra de sempre: as configurações (velocidade, tempo, quantidade de pulos) são variáveis públicas e aparecem no **Inspector**. Quer mudar o jogo? Mude só esses campos, sem mexer no código.

Etapa 1: o script novo do Player

Por quê: a sensação de controle é o que faz um jogo de plataforma ser bom ou ruim. Um player que já sai correndo no máximo fica difícil de controlar em plataformas pequenas, um que nunca acelera deixa os trechos longos chatos. Com estados de movimento, deslizamento, pulos configuráveis e coyote time, dá para ajustar tudo isso pelo Inspector.

Como fazer:

- Antes de trocar o código, anote os valores que estão no Inspector do Player (velocidade, força do pulo, sensibilidade do mouse). O campo `velocidade` deixa de existir e vira dois: `velocidadeAndar` e `velocidadeCorrer`.
- Abra o script `Player` e troque **todo** o conteúdo por este:

```
using UnityEngine;

// Estados possíveis do movimento do player
public enum EstadoMovimento
{
```

```
    Parado,
    Andando,
    Correndo,
    Deslizando   // soltou as teclas, mas ainda está escorregando por causa do
deslizamento
}

public class Player : MonoBehaviour
{
    [Header("Andar / Correr")]
    [Tooltip("Velocidade enquanto o player está andando (unidades por segundo)")]
    public float velocidadeAndar = 5f;

    [Tooltip("Velocidade quando o player entra no estado de corrida")]
    public float velocidadeCorrer = 9f;

    [Tooltip("Tempo (em segundos) andando sem parar até virar corrida. 0 = corre
na hora")]
    public float tempoParaCorrer = 1.5f;

    [Tooltip("Ligado: a velocidade vai subindo aos poucos enquanto anda até virar
corrida.\nDesligado: anda na velocidade normal e muda de uma vez para a de
corrida.")]
    public bool aumentoGradual = true;

    [Tooltip("Formato da aceleração quando o Aumento Gradual está ligado.\n1 =
sobe por igual (linear)\nMaior que 1 = começa devagar e acelera no final (ex:
2)\nMenor que 1 = acelera rápido no começo e suaviza no final (ex: 0.5)")]
    [Min(0.1f)]
    public float curvaAceleracao = 1f;

    [Tooltip("Ligado: só acumula tempo de corrida enquanto segura a tecla de
correr.\nDesligado: basta andar sem parar.")]
    public bool exigirTeclaCorrer = false;

    public KeyCode teclaCorrer = KeyCode.LeftShift;

    [Header("Deslizamento")]
    [Tooltip("Quanto o movimento \"desliza\" ao começar, parar e mudar de
direção.\n0 = sem deslizamento (para e arranca na hora)\n0.1 = suave\n0.3 ou mais
= parece gelo")]
    [Range(0f, 1f)]
    public float deslizamento = 0.1f;

    private EstadoMovimento estadoAtual = EstadoMovimento.Parado; // estado atual
do movimento
    private float velocidadeAtual = 0f;                           // velocidade
que está sendo usada agora

    public EstadoMovimento EstadoAtual => estadoAtual; // outros scripts (ex:
animação) podem ler o estado

    // ===== Informações para outros scripts (ex: UI de debug) =====
    // Velocidade REAL no chão (X e Z), já contando o deslizamento
    public float VelocidadeAtual => rb != null ? new Vector3(rb.velocity.x, 0f,
rb.velocity.z).magnitude : 0f;
    public bool EstaNoChao => contatosNoChao > 0;

    // Quantos pulos o player consegue dar AGORA
```

```
    public int PulosDisponiveis
    {
        get
        {
            // No chão ou dentro do coyote time (sem ter pulado): tem todos os
pulos
            bool podePularDoChao = (Time.time - ultimoTempoNoChao <= tempoCoyote)
&& !pulouDoChao;
            return podePularDoChao ? pulosMaximos : pulosNoAr;
        }
    }

    // Quanto tempo de coyote time ainda resta
    public float CoyoteRestante
    {
        get
        {
            if (EstaNoChao) return tempoCoyote;   // no chão: janela cheia
            if (pulouDoChao) return 0f;           // já pulou: coyote não vale
mais
            return Mathf.Max(0f, tempoCoyote - (Time.time - ultimoTempoNoChao));
        }
    }

    [Header("Pulo")]
    public float forcaPulo = 6f;       // Força aplicada verticalmente ao pular

    [Range(0, 2)]
    [Tooltip("0 = não pula, 1 = pulo normal (padrão), 2 = pulo duplo")]
    public int pulosMaximos = 1;

    [Tooltip("Coyote time: tempo (em segundos) que o player ainda pode dar o pulo
do chão depois de sair de uma borda")]
    public float tempoCoyote = 0.15f;

    [Header("Rotação do Mouse")]
    public float sensibilidadeMouse = 200f;   // Quanto o mouse influencia a
rotação (quanto maior, mais sensível)
    public Transform cameraPersonagem;        // Referência da câmera (deve ser
filha do personagem na Hierarchy)
    public float limiteAnguloVertical = 80f;  // Limite máximo de ângulo para
olhar pra cima/baixo (evita giro de 360°)

    private Rigidbody rb;             // Referência ao componente Rigidbody do
personagem, usado para física e movimento
    private float rotacaoVertical = 0f; // Guarda o ângulo vertical atual da
câmera (acumulado a cada frame)
    private int contatosNoChao = 0;    // Conta quantas plataformas o personagem
está tocando ao mesmo tempo (usado para permitir o pulo)

    private int pulosNoAr = 0;                // quantos pulos ainda podem ser
usados no ar
    private float proximaTrocaPermitida = 0f; // momento (Time.time) a partir do
qual pode trocar os pulos de novo

    private float ultimoTempoNoChao = -999f; // COYOTE: último momento em que o
player estava tocando o chão
    private bool pulouDoChao = false;        // COYOTE: impede usar o pulo do chão
duas vezes
```

```
    private float tempoMovendo = 0f;          // CORRIDA: há quanto tempo o player
está andando sem parar

    void Start()
    {
        rb = GetComponent<Rigidbody>(); // Pega o Rigidbody que está no mesmo
GameObject deste script
        // Trava o cursor do mouse no centro da tela e o deixa invisível (padrão
em jogos FPS/terceira pessoa)
        Cursor.lockState = CursorLockMode.Locked;
        Cursor.visible = false;

        // Se já existe um checkpoint salvo (sobrevive ao reload de cena),
        // reposiciona o player nele em vez de nascer no ponto padrão da cena.
        if (CheckpointManager.posicaoCheckpoint.HasValue)
        {
            transform.position = CheckpointManager.posicaoCheckpoint.Value;
            transform.rotation = Quaternion.Euler(0f,
CheckpointManager.rotacaoYCheckpoint, 0f);
        }
    }

    void Update()
    {
        // Rotação é feita no Update porque o mouse é lido a cada frame
renderizado (mais suave visualmente)
        RotacionarComMouse();

        // COYOTE: enquanto estiver no chão, vai atualizando o "último momento no
chão"
        if (contatosNoChao > 0)
            ultimoTempoNoChao = Time.time;

        // COYOTE: conta como "no chão" se está tocando o chão OU saiu dele há
menos de tempoCoyote segundos
        bool podePularDoChao = (Time.time - ultimoTempoNoChao <= tempoCoyote) &&
!pulouDoChao;

        if (Input.GetKeyDown(KeyCode.Space))
        {
            if (podePularDoChao && pulosMaximos > 0)
            {
                // Pulo do chão (ou coyote): gasta 1 pulo e sobram (pulosMaximos -
1) para usar no ar
                Pular();
                pulosNoAr = pulosMaximos - 1;
                pulouDoChao = true; // só volta a ser false quando encostar no
chão de novo
            }
            else if (pulosNoAr > 0)
            {
                // Pulo no ar (ex: segundo pulo do pulo duplo)
                Pular();
                pulosNoAr--;
            }
        }

        // Tecla de escape (ESC): libera o cursor do mouse
```

```
        if (Input.GetKeyDown(KeyCode.Escape))
        {
            Cursor.lockState = CursorLockMode.None;
            Cursor.visible = true;
        }
    }

    void FixedUpdate()
    {
        // Movimento é feito no FixedUpdate porque envolve física (Rigidbody),
        MovimentarTeclas();
    }

    // Aplica o pulo
    void Pular()
    {
        // Zera a velocidade vertical antes de pular, assim o pulo duplo tem
sempre a mesma altura
        Vector3 v = rb.velocity;
        v.y = 0f;
        rb.velocity = v;

        // Aplica uma força instantânea para cima (ForceMode.VelocityChange ignora
a massa do objeto)
        rb.AddForce(Vector3.up * forcaPulo, ForceMode.VelocityChange);
    }

    // Chamado pela ZonaDePulo. Troca a quantidade de pulos se não estiver em
cooldown.
    // Retorna true se a troca aconteceu.
    public bool DefinirPulos(int quantidade, float cooldown)
    {
        if (Time.time < proximaTrocaPermitida)
            return false;

        pulosMaximos = Mathf.Clamp(quantidade, 0, 2);
        pulosNoAr = pulosMaximos; // redefine os pulos na hora
        proximaTrocaPermitida = Time.time + cooldown;
        return true;
    }

    // Lê as teclas WASD e movimenta o personagem com base na direção que ele está
olhando
    void MovimentarTeclas()
    {
        Vector3 direcao = Vector3.zero; // Vetor que vai acumular a direção do
movimento

        // transform.forward/right são baseados na rotação atual do personagem,
        // ou seja, "frente" sempre é para onde ele está olhando
        if (Input.GetKey(KeyCode.W))
            direcao += transform.forward;  // Anda para frente
        if (Input.GetKey(KeyCode.S))
            direcao -= transform.forward;  // Anda para trás
        if (Input.GetKey(KeyCode.A))
            direcao -= transform.right;    // Anda para a esquerda
        if (Input.GetKey(KeyCode.D))
            direcao += transform.right;    // Anda para a direita
```

```
        // Descobre a velocidade de acordo com o estado (parado / andando / correndo)
        AtualizarEstado(direcao != Vector3.zero);

        // Normalized garante que andar na diagonal (ex: W+D) não seja mais rápido que andar reto
        // Essa é a velocidade que o player QUER ter (o "alvo")
        Vector3 alvo = direcao.normalized * velocidadeAtual;

        // Velocidade horizontal que o player TEM agora (sem o Y, que é queda/pulo)
        Vector3 atual = new Vector3(rb.velocity.x, 0f, rb.velocity.z);

        // DESLIZAMENTO: em vez de pular direto para o alvo, chega perto dele um pouco a cada passo
        Vector3 movimento;
        if (deslizamento <= 0f)
        {
            movimento = alvo; // sem deslizamento: muda na hora, como antes
        }
        else
        {
            // Quanto do caminho até o alvo é feito neste passo (0 a 1)
            // Deslizamento maior = passo menor = mudança mais lenta e suave
            float passo = Mathf.Clamp01(Time.fixedDeltaTime / deslizamento);
            movimento = Vector3.Lerp(atual, alvo, passo);
        }

        // Mantém a velocidade vertical atual (queda/pulo) e só altera o movimento horizontal (X e Z)
        movimento.y = rb.velocity.y;

        // Aplica a velocidade final diretamente no Rigidbody
        rb.velocity = movimento;
    }

    // CORRIDA: decide o estado atual e a velocidade que deve ser usada
    void AtualizarEstado(bool estaMovendo)
    {
        // Se exigir a tecla de correr e ela não estiver pressionada, não acumula tempo de corrida
        bool podeAcumularCorrida = !exigirTeclaCorrer || Input.GetKey(teclaCorrer);

        if (!estaMovendo)
        {
            // Soltou as teclas: zera o tempo de corrida e o alvo de velocidade
            tempoMovendo = 0f;
            velocidadeAtual = 0f;

            // DESLIZANDO: sem teclas, mas o corpo ainda está se mexendo no chão (X e Z)
            // Abaixo de 0.1 já conta como parado (a velocidade nunca chega exatamente a 0)
            if (VelocidadeAtual > 0.1f)
                estadoAtual = EstadoMovimento.Deslizando;
            else
                estadoAtual = EstadoMovimento.Parado;
            return;
```

```
        }

        if (podeAcumularCorrida)
            tempoMovendo += Time.fixedDeltaTime; // continua andando: soma o tempo
        else
            tempoMovendo = 0f;                   // soltou a tecla de correr:
volta a andar

        // Progresso de 0 (acabou de começar a andar) até 1 (virou corrida)
        float progresso = tempoParaCorrer <= 0f ? 1f : Mathf.Clamp01(tempoMovendo
/ tempoParaCorrer);

        if (aumentoGradual)
        {
            // Velocidade sobe aos poucos; elevar o progresso à curva muda o
"formato" da subida
            float t = Mathf.Pow(progresso, curvaAceleracao);
            velocidadeAtual = Mathf.Lerp(velocidadeAndar, velocidadeCorrer, t);
        }
        else
        {
            // Anda na velocidade normal e só troca quando o tempo completa
            velocidadeAtual = progresso >= 1f ? velocidadeCorrer :
velocidadeAndar;
        }

        estadoAtual = progresso >= 1f ? EstadoMovimento.Correndo :
EstadoMovimento.Andando;
    }

    // Controla a rotação do personagem e da câmera com base no movimento do mouse
    void RotacionarComMouse()
    {
        // Pega o quanto o mouse se moveu no eixo X (horizontal) e Y (vertical)
desde o último frame
        float mouseX = Input.GetAxis("Mouse X") * sensibilidadeMouse *
Time.deltaTime;
        float mouseY = Input.GetAxis("Mouse Y") * sensibilidadeMouse *
Time.deltaTime;

        // Rotaciona o personagem inteiro no eixo Y (esquerda/direita)
        transform.Rotate(Vector3.up * mouseX);

        // Só mexe na rotação vertical se a câmera estiver configurada no
Inspector
        if (cameraPersonagem != null)
        {
            // Subtrai o mouseY: mover o mouse pra cima deve olhar pra cima
(rotação negativa no eixo X)
            rotacaoVertical -= mouseY;

            // Limita o ângulo vertical para não deixar o personagem "olhar de
cabeça para baixo"
            rotacaoVertical = Mathf.Clamp(rotacaoVertical, -limiteAnguloVertical,
limiteAnguloVertical);

            // Aplica a rotação SOMENTE na câmera (localRotation), não no
personagem inteiro
            cameraPersonagem.localRotation = Quaternion.Euler(rotacaoVertical, 0f,
```

```
0f);
        }
    }

    // Chamado automaticamente pela Unity quando o Collider deste objeto ENCOSTA
em outro
    void OnCollisionEnter(Collision colisao)
    {
        if (colisao.gameObject.CompareTag("Plataforma"))
        {
            contatosNoChao++; // Soma 1 contato: cada plataforma tocada conta
separadamente

            // Ao pisar no chão, recarrega os pulos de ar
            pulosNoAr = Mathf.Max(0, pulosMaximos - 1);

            // COYOTE: tocou o chão, libera o pulo do chão de novo
            pulouDoChao = false;
        }
    }

    // Chamado automaticamente pela Unity quando o Collider deste objeto não está
tocando no outro
    void OnCollisionExit(Collision colisao)
    {
        if (colisao.gameObject.CompareTag("Plataforma"))
        {
            contatosNoChao--; // Remove esse contato; o pulo só é bloqueado quando
o contador chegar a 0
        }
    }
}
```

Etapa 2: entendendo o script por partes

O script cresceu, mas continua sendo feito de pedaços pequenos. Cada um cuida de uma coisa só.

Estados do movimento

- `enum EstadoMovimento` é uma lista de nomes possíveis: `Parado`, `Andando`, `Correndo` e `Deslizando`. É mais fácil de ler do que usar números como 0, 1, 2 e 3.
- `estadoAtual` guarda em qual desses estados o player está agora.
- `Deslizando` aparece quando o jogador solta as teclas, mas o player ainda está escorregando. Quando a velocidade fica menor que 0.1, ele vira `Parado`. Esse limite existe porque, com o deslizamento, a velocidade vai diminuindo cada vez mais devagar e quase nunca chega exatamente a 0.

Deslizamento

- `deslizamento` é quanto o movimento "escorrega" ao começar, parar e mudar de direção. O `[Range(0f, 1f)]` vira uma barrinha no Inspector.
- `alvo` é a velocidade que o player **quer** ter (para onde as teclas mandam). `atual` é a velocidade que ele **tem** agora, sem contar o Y (queda e pulo).

- Sem deslizamento, o player pula direto para o alvo: para seco e arranca na hora. Com deslizamento, `Vector3.Lerp` faz ele chegar só um pedaço mais perto do alvo a cada passo da física.
- `passo` é o tamanho desse pedaço. Quanto maior o deslizamento, menor o passo, e mais devagar a velocidade muda.
- Com 0 o movimento fica igual a antes. Entre 0.05 e 0.15 fica suave, mas ainda preciso. Com 0.3 ou mais parece gelo.

Andar e correr

- `velocidadeAndar` e `velocidadeCorrer` são as duas velocidades.
- `tempoParaCorrer` é quanto tempo o player precisa andar sem parar até virar corrida. Com 0, ele já começa correndo.
- `aumentoGradual` decide como a troca acontece. Ligado: a velocidade vai subindo aos poucos. Desligado: anda normal e troca de uma vez quando o tempo completa.
- `curvaAceleracao` muda o formato dessa subida. 1 é uma subida por igual. Maior que 1 (ex: 2) começa devagar e acelera no final, como quem "pega embalo". Menor que 1 (ex: 0.5) ganha velocidade rápido no começo.
- `exigirTeclaCorrer` e `teclaCorrer`: se ligado, o tempo de corrida só conta enquanto o jogador segura o Shift (ou a tecla escolhida).

A função AtualizarEstado

- `tempoMovendo` é um cronômetro: soma o tempo enquanto o player anda e zera quando ele para.
- `progresso` transforma esse tempo num número de 0 (acabou de começar a andar) até 1 (virou corrida). `Mathf.Clamp01` impede que passe de 1.
- `Mathf.Pow(progresso, curvaAceleracao)` aplica a curva, e `Mathf.Lerp` acha a velocidade entre andar e correr. Lerp é "o meio do caminho": com 0 dá a velocidade de andar, com 1 a de correr, com 0.5 a metade.
- Como o pulo não para o cronômetro, pular durante a corrida não faz o player perder o embalo.

Pulos e pulo duplo

- `pulosMaximos` é quantos pulos o player tem: 0, 1 ou 2. O `[Range(0, 2)]` vira uma barrinha no Inspector e não deixa colocar outro número.
- `pulosNoAr` conta quantos pulos ainda sobram no ar. Pular do chão gasta 1; com pulo duplo, sobra 1 para usar no ar.
- `Pular()` zera a velocidade vertical antes de pular. Sem isso, o segundo pulo, dado enquanto o player cai, sairia bem mais fraco que o primeiro.
- `DefinirPulos` é a função que a zona de pulo chama. Ela é `public` justamente para outro script poder usar.

Coyote time

- O nome vem do Coiote dos desenhos animados, que sai correndo do penhasco e só cai depois de alguns instantes.
- `ultimoTempoNoChao` guarda o último momento em que o player encostou no chão. `Time.time` é o relógio do jogo, em segundos.
- O player pode dar o pulo do chão se está no chão **ou** se saiu dele há menos de `tempoCoyote` segundos.
- `pulouDoChao` impede de usar o pulo do chão duas vezes: sem ele, o jogador poderia pular e, ainda dentro da janela de coyote, pular de novo.

Informações para outros scripts

- `VelocidadeAtual`, `EstaNoChao`, `PulosDisponiveis` e `CoyoteRestante` são propriedades: outros scripts conseguem ler, mas não conseguem mudar. O painel de informações vai usá-las.
- `VelocidadeAtual` lê a velocidade real do Rigidbody no chão (X e Z). Por isso, ao soltar as teclas, o painel mostra o número caindo aos poucos durante o deslizamento.
- A seta `=>` é um jeito curto de escrever uma propriedade que só devolve um valor.

Valores para começar os testes:

Campo	Valor inicial	O que muda
Velocidade Andar	5	Rapidez ao começar a andar
Velocidade Correr	9	Rapidez máxima
Tempo Para Correr	1.5	Quanto demora para virar corrida
Curva Aceleracao	1	Formato da subida de velocidade
Deslizamento	0.1	Suavidade ao arrancar, parar e virar
Forca Pulo	6	Altura do pulo
Pulos Maximos	1	Quantidade de pulos (0, 1 ou 2)
Tempo Coyote	0.15	Janela de pulo depois da borda

Etapa 3: a zona de pulo

Por quê: mudar a quantidade de pulos no meio da fase cria desafios novos sem precisar de inimigos: um trecho onde o player não pode pular, outro onde ele ganha pulo duplo para alcançar uma plataforma alta.

Como fazer:

- Crie um **3D Object > Cube** e dê o nome de `ZonaPuloDuplo`. Estique no formato de uma parede que o player precisa atravessar, por exemplo Scale 0.5, 4, 4.
- No **Box Collider**, marque **Is Trigger**. Assim o player atravessa a parede e o Unity só avisa que ele passou.
- Use uma cor para cada tipo de zona, por exemplo verde para 2 pulos, branco para 1 e vermelho para 0. Um material transparente deixa com cara de portal.
- Crie um script chamado `ZonaDePulo`, coloque na parede e cole o código:

```
using UnityEngine;

// Coloque este script em uma parede/objeto com Collider marcado como "Is
Trigger".
// Quando o Player encosta, ele passa a ter a quantidade de pulos definida aqui.
public class ZonaDePulo : MonoBehaviour
{
    [Header("Configuração de Pulo")]
    [Range(0, 2)]
    public int quantidadePulos = 1;     // 0 = não pode pular, 1 = pulo normal
(padrão), 2 = pulo duplo

    [Tooltip("Tempo (em segundos) que o player precisa esperar antes de poder
trocar a quantidade de pulos de novo")]
    public float cooldown = 2f;

    // Chamado uma vez quando a fase começa
    void Start()
    {
        // Se a zona tiver a tag "Limite", ela fica invisível durante o jogo.
        // Só o desenho some: o collider continua funcionando normalmente.
        if (CompareTag("Limite"))
        {
            Renderer desenho = GetComponent<Renderer>();
            if (desenho != null)
                desenho.enabled = false;
        }
    }

    // Chamado automaticamente quando algo ENTRA no trigger
    void OnTriggerEnter(Collider outro)
    {
        // Procura o script Player no objeto que entrou (ou nos "pais" dele, caso
o collider esteja num filho)
        Player player = outro.GetComponentInParent<Player>();

        if (player != null)
        {
            // Tenta trocar a quantidade de pulos. Se o player ainda estiver em
cooldown, nada acontece.
            bool trocou = player.DefinirPulos(quantidadePulos, cooldown);

            if (trocou)
                Debug.Log("Player agora tem " + quantidadePulos + " pulo(s)");
        }
    }
}
```

Entendendo o código:

- `quantidadePulos` é quantos pulos o player passa a ter depois de atravessar esta parede.
- `cooldown` é o tempo que o player precisa esperar antes de outra zona poder mudar os pulos dele de novo.
- `OnTriggerEnter` é chamado sozinho pelo Unity quando algo entra no trigger. Ele só funciona porque o Player tem um **Rigidbody**.
- `GetComponentInParent<Player>()` procura o script Player em quem entrou. Se não achar, é outro objeto qualquer, e nada acontece.
- A zona não muda os pulos sozinha: ela pede para o Player com `DefinirPulos`. Quem decide se aceita (por causa do cooldown) é o Player.
- `Start` roda uma vez quando a fase começa. Se a zona tiver a tag `Limite`, ele desliga o `Renderer` (o desenho da peça). A zona some da tela, mas o collider continua lá e continua funcionando.

Zona invisível com a tag Limite:

- No topo do Inspector da zona, abra **Tag > Add Tag...**, clique no **+** e crie a tag `Limite`, escrita exatamente assim, com L maiúsculo.
- Volte na zona e escolha a tag `Limite`. No editor ela continua aparecendo, para você conseguir posicionar. Quando a fase começa, ela fica invisível.
- Sem a tag, a zona continua visível durante o jogo, com a cor que você escolheu.

O cooldown fica guardado no Player, e não na zona. Por isso ele vale para **todas** as zonas: depois de uma troca, nenhuma outra zona funciona até o tempo acabar. Ao atravessar uma zona, os pulos também são recarregados na hora, mesmo se o player estiver no ar.

Etapa 4: o painel de informações

Por quê: é difícil ajustar o que não dá para ver. Com o painel na tela, o jogador vê na hora quando a corrida começa, quantos pulos sobraram e quanto tempo de coyote ainda resta. É uma ferramenta de teste, como o Output.

Como fazer:

- Na Hierarchy, clique com o botão direito e escolha **UI > Text - TextMeshPro**. Se o Unity pedir para importar o TMP Essentials, clique em **Import TMP Essentials**. Isso cria um **Canvas** com o texto dentro.
- Selecione o **Canvas** e confira se **Render Mode** está em **Screen Space - Overlay**. Nesse modo o texto fica sempre por cima de tudo.
- Posicione o texto no canto superior esquerdo e aumente a altura da caixa para caber 5 linhas.
- Crie um script chamado `PlayerUI`, coloque no Canvas e cole o código:

```
using UnityEngine;
using TMPro;
```

```
// Mostra na tela as informações do Player (velocidade, estado, pulos, coyote, chão).
// Coloque este script em qualquer objeto (ex: no próprio Canvas) e arraste as referências no Inspector.
public class PlayerUI : MonoBehaviour
{
    public Player player;     // Arraste o objeto do Player aqui
    public TMP_Text texto;    // Arraste o texto (TextMeshPro) do Canvas aqui

    void Update()
    {
        // Se faltar alguma referência, não faz nada (evita erro no Console)
        if (player == null || texto == null)
            return;

        texto.text =
            "Velocidade: " + player.VelocidadeAtual.ToString("F1") + "\n" +
            "Estado: " + player.EstadoAtual + "\n" +
            "Pulos: " + player.PulosDisponiveis + " / " + player.pulosMaximos + "\n" +
            "Coyote: " + player.CoyoteRestante.ToString("F2") + "s / " + player.tempoCoyote.ToString("F2") + "s\n" +
            "No chão: " + (player.EstaNoChao ? "Sim" : "Não");
    }
}
```

Entendendo o código:

- `using TMPro;` libera o uso dos textos do TextMeshPro no script.
- `player` e `texto` são arrastados no Inspector: o Player da cena e o texto do Canvas.
- O `if` do começo impede erros no Console se alguém esquecer de arrastar uma das referências.
- `ToString("F1")` mostra o número com 1 casa depois da vírgula, e `"F2"` com 2. Sem isso, a velocidade apareceria como 7.236841.
- `"\n"` é uma quebra de linha: cada informação fica em uma linha.
- `? "Sim" : "Não"` é um if curto: se estiver no chão escreve Sim, senão escreve Não.

O painel vai ficar assim durante o jogo:

```
Velocidade: 7.2
Estado: Andando
Pulos: 1 / 2
Coyote: 0.00s / 0.15s
No chão: Não
```

Etapa 5: montar a fase de testes

Por quê: testar a movimentação no meio da fase de verdade mistura tudo: o jogador não sabe se errou o pulo porque o número está ruim ou porque a fase está difícil. A fase de testes é um "laboratório", com uma estação para cada mecânica.

Como fazer:

- Crie uma cena nova chamada `FaseTestes` (**File > New Scene**) e coloque nela o Player, a câmera e o Canvas com o painel. Se o Player já for um Prefab, é só arrastar.
- Todo chão e toda plataforma precisam da tag `Plataforma`. Sem ela, o player não sabe que está no chão e não pula.

Etapa 6: ajustar a movimentação

Por quê: os números da Etapa 2 são só um ponto de partida. Cada jogo pede um tipo de movimento: um jogo de plataforma de precisão quer um player lento e controlável; um jogo de corrida quer acelerar rápido. O objetivo de hoje é usar a fase de testes e o painel para achar os valores que combinam com o **seu** jogo.

Como fazer:

- Mude **um valor por vez**. Se mudar três coisas juntas, não dá para saber qual delas melhorou ou piorou.
- Dê Play, passe pela estação daquele valor e olhe o painel: a corrida começa na hora certa? O pulo alcança a plataforma? O coyote está ajudando ou parece trapaça?
- Use os vãos crescentes para descobrir a distância máxima do pulo andando e correndo. Isso define o tamanho dos buracos que você pode colocar na fase de verdade.
- Anote os valores que funcionaram e o que mudou na sensação de jogar.

Dicas de valores para experimentar:

Quero que o player...	**Mude**	**Para**
demore mais para correr	Tempo Para Correr	Um valor maior, como 2.5
pegue embalo aos poucos	Curva Aceleracao	Maior que 1, como 2
acelere rápido e depois suavize	Curva Aceleracao	Menor que 1, como 0.5
troque para corrida de uma vez	Aumento Gradual	Desligado
só corra segurando Shift	Exigir Tecla Correr	Ligado
pare e vire com precisão	Deslizamento	0 ou perto de 0.05
tenha um movimento mais suave	Deslizamento	Entre 0.1 e 0.15
pareça andar no gelo	Deslizamento	0.3 ou mais
perdoe mais os pulos atrasados	Tempo Coyote	Entre 0.15 e 0.2

Tudo o que você muda no Inspector **durante o Play** é desfeito quando o jogo para. Para não perder um valor bom: com o jogo rodando, clique nos três pontinhos do componente Player e escolha **Copy Component**. Pare o jogo, clique de novo e escolha **Paste Component Values**.

Etapa 7: dicas de game design para a movimentação

Por quê: na aula de game design e level design vimos a regra de ouro do plataforma 3D: primeiro o personagem tem que ser bom de controlar numa sala vazia, e só depois vale a pena construir fases. A fase de testes de hoje é exatamente essa sala. Estas dicas ligam as mecânicas novas às ideias daquela aula.

O loop curto vem primeiro

- O loop curto (ver o obstáculo → pular → cair na plataforma) é o que o jogador repete centenas de vezes. Se ele não for gostoso, nenhuma fase bonita segura o jogador.
- O personagem precisa responder na hora que o botão é apertado. Deslizamento alto deixa o movimento bonito, mas atrasado: se o jogador sente que o player "não obedece", diminua o valor.
- O coyote time é a "ajudinha" que vimos na aula: o jogador acha que pulou na hora certa, e o jogo concorda com ele. Ninguém percebe que ele existe, mas todo mundo percebe quando ele falta.

Meça o pulo antes de montar a fase

- Use os vãos crescentes e a escada de alturas para anotar até onde o player pula andando e correndo.
- Todo buraco da fase de verdade tem que caber nessa medida. Se a corrida demora para começar, lembre que o jogador pode não ter espaço para pegar embalo antes de um buraco.
- Quando mudar a velocidade, a força do pulo ou a curva de aceleração, meça de novo. As medidas antigas deixam de valer.

Ensine a zona de pulo em 4 passos

- **Apresentar**: a primeira zona de pulo duplo aparece em cima do chão. Se o jogador errar, não acontece nada.
- **Praticar**: agora o pulo duplo é preciso para passar por cima de um buraco.
- **Surpreender**: misture com outra mecânica, por exemplo uma zona de 0 pulos logo depois, ou um pulo duplo que precisa do coyote time.
- **Fechar**: um último uso, mais fácil, antes do fim da fase.

Coesão visual: o jogador precisa saber quantos pulos tem

- Uma cor, um significado. Escolha uma cor para cada quantidade de pulos e use sempre igual, no jogo inteiro.
- Se a zona não aparece, o jogador não sabe que os pulos mudaram. Ele aperta o espaço, nada acontece, e ele acha que o jogo está com problema. Isso quebra a regra "se parece chão, tem que ser chão".
- Use a tag `Limite` só quando o jogador já sabe o que vai acontecer, ou quando o cenário avisa de outro jeito: um piso de cor diferente, partículas no ar, uma placa, uma porta.
- Na fase de testes, deixe as zonas visíveis. Assim fica fácil ver onde cada uma está.

Zona	Cor sugerida	Significado
0 pulos	Vermelho	Aqui não dá para pular
1 pulo	Branco	Volta ao normal
2 pulos	Verde	Ganha pulo duplo

Ritmo: correr também é uma ação

- Variar a ação deixa a fase menos cansativa. Uma reta longa onde o player chega na corrida é um bom "lugar para respirar" entre dois trechos de pulos precisos.
- Plataformas pequenas pedem andar, não correr. Se o trecho seguinte é de precisão, coloque o desafio logo depois de uma curva ou de um degrau, onde o jogador naturalmente perde o embalo.
- Deslizamento alto pode virar uma mecânica de fase, como uma área de gelo. Se fizer isso, deixe o chão com cara de gelo (azul claro e brilhante): o visual tem que explicar a regra.

Teste com um colega

- Fique quieto. Se você precisar explicar que ali tem um pulo duplo, a fase precisa mudar, não o jogador.
- Repare onde ele morreu. Três mortes no mesmo lugar quase sempre é problema da fase, ou de um número da movimentação.
- Mude uma coisa de cada vez. Assim dá para saber o que realmente resolveu o problema.
- Facilite antes de dificultar. Comece com o coyote time mais generoso e o deslizamento baixo, e endureça depois que o controle estiver bom.

Testar e problemas comuns

Dê **Play** na fase de testes e confira:

- Ao começar a andar, o estado muda para Andando e, depois do tempo configurado, para Correndo
- Ao soltar as teclas, o estado passa por Deslizando e depois volta para Parado
- Durante o deslizamento, a velocidade no painel cai aos poucos até 0
- Uma zona com a tag Limite some quando a fase começa, mas continua mudando os pulos
- Ao atravessar uma zona, a quantidade de pulos no painel muda
- Com 2 pulos, o player consegue pular de novo no ar
- Com 0 pulos, o espaço não faz nada
- Ao sair de uma borda sem pular, o coyote no painel desce até 0 e, antes disso, ainda dá para pular

Problema	Causa provável	Solução

O player não pula	O chão não tem a tag `Plataforma`, ou Pulos Maximos está em 0	Coloque a tag no chão e confira os pulos no painel
Os valores de velocidade sumiram	O campo `velocidade` foi trocado por `velocidadeAndar` e `velocidadeCorrer`	Coloque de novo os valores que você anotou
O player nunca corre	**Exigir Tecla Correr** está ligado, ou o tempo para correr está muito alto	Segure Shift ou desligue a opção; diminua o tempo
A zona de pulo não faz nada	**Is Trigger** está desmarcado, ou o player ainda está no cooldown	Marque Is Trigger e espere o cooldown acabar
O player bate na zona como se fosse parede	**Is Trigger** está desmarcado	Marque Is Trigger no Box Collider da zona
A zona com a tag Limite continua aparecendo	A tag está escrita diferente, ou a zona não tem o script novo	Confira se a tag é `Limite` e se o script tem o `Start`
O player parece atrasado ou "não obedece"	O deslizamento está alto	Diminua o deslizamento para perto de 0.1
O estado nunca sai de Deslizando	O player está sendo empurrado por outro objeto, ou está no ar se movendo para frente	É o esperado: ele está se mexendo sem o jogador apertar nada
Erro: TMP_Text could not be found	O TextMeshPro não foi importado	Window > TextMeshPro > Import TMP Essential Resources
Objetos ficam na frente do painel	O Canvas não está em Screen Space - Overlay	Troque o Render Mode do Canvas
O painel não muda	O Player ou o texto não foram arrastados no PlayerUI	Arraste os dois no Inspector
O player gruda na parede ao pular contra ela	O atrito do collider segura o player	Crie um Physic Material com Friction 0 e coloque no collider do Player
Mudei um valor e ele voltou	O valor foi mudado durante o Play	Use Copy Component e Paste Component Values

Próximas aulas

- **Câmera que reage à corrida**: o campo de visão abre um pouco quando o player corre, para dar sensação de velocidade.
- Correção de bugs.
- Menus aprimorados

Tarefa em aula

- **Faça toda a implementação do arquivo**
- **Monte a fase de testes**
- **Use pelo menos 2 zonas de pulo diferentes (0, 1 e 2 pulos)**
- **Monte um trecho que ensine a zona de pulo duplo nos 4 passos (apresentar, praticar, surpreender, fechar)**
- **Peça para um colega jogar sem explicar nada e observe onde ele trava**
- **Desafio: monte um trecho que só dá para passar usando coyote time e pulo duplo juntos**